

Лабораторная работа №7

Вариант 7

1 Задание 1. Аналитическое исследование функции

Постановка задачи

Дано по варианту:

$$f(x, y) = 10^{-2} (8x^2 + 2xy + 35x + 8y + 12), \quad (x, y) \in [-20, 20] \times [-50, 50].$$

Требуется:

1. найти стационарные точки;
2. определить, есть ли седловые точки и локальные экстремумы;
3. найти глобальный минимум на заданной области;
4. построить линии уровня функции.

Область определения функции без ограничения:

$$D(f) = \mathbb{R}^2.$$

Область определения с учетом ограничения:

$$[-20, 20] \times [-50, 50].$$

Поиск стационарной точки

Найдем частные производные первого порядка:

$$f'_x = 10^{-2}(16x + 2y + 35),$$

$$f'_y = 10^{-2}(2x + 8).$$

Стационарные точки определяются из системы:

$$\begin{cases} f'_x = 0, \\ f'_y = 0. \end{cases}$$

Подставим найденные производные:

$$\begin{cases} 10^{-2}(16x + 2y + 35) = 0, \\ 10^{-2}(2x + 8) = 0. \end{cases}$$

Так как множитель $10^{-2} \neq 0$, получаем равносильную систему:

$$\begin{cases} 16x + 2y + 35 = 0, \\ 2x + 8 = 0. \end{cases}$$

Из второго уравнения:

$$2x + 8 = 0, \quad x = -4.$$

Подставим $x = -4$ в первое уравнение:

$$16(-4) + 2y + 35 = 0,$$

$$-64 + 2y + 35 = 0,$$

$$2y - 29 = 0, \quad y = 14.5.$$

Следовательно, единственная стационарная точка:

$$\boxed{(-4, 14.5)}.$$

Определение типа стационарной точки

Найдем частные производные второго порядка:

$$f''_{xx} = 10^{-2} \cdot 16 = 0.16,$$

$$f''_{xy} = f''_{yx} = 10^{-2} \cdot 2 = 0.02,$$

$$f''_{yy} = 0.$$

Гессиан:

$$H = \begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial y \partial x} \\ \frac{\partial^2 f}{\partial x \partial y} & \frac{\partial^2 f}{\partial y^2} \end{pmatrix} = 10^{-2} \begin{pmatrix} 16 & 2 \\ 2 & 0 \end{pmatrix} = \begin{pmatrix} 0.16 & 0.02 \\ 0.02 & 0 \end{pmatrix}.$$

Определитель гессиана:

$$D = \begin{vmatrix} 0.16 & 0.02 \\ 0.02 & 0 \end{vmatrix} = 0.16 \cdot 0 - 0.02 \cdot 0.02 = -0.0004.$$

Так как

$$D < 0,$$

то гессиан является неопределенным. Значит, стационарная точка является седловой.

Итак:

$$\boxed{(-4, 14.5) \text{ — седловая точка.}}$$

Локальных минимумов и максимумов внутри области нет, так как единственная стационарная точка является седловой.

Поиск глобального минимума

Найдем глобальный минимум функции на прямоугольной области:

$$[-20, 20] \times [-50, 50].$$

Область является замкнутой и ограниченной, а функция непрерывна, поэтому глобальный минимум и глобальный максимум на этой области существуют.

Функцию удобно записать так:

$$f(x, y) = 10^{-2} (8x^2 + 35x + 12 + y(2x + 8)).$$

При фиксированном x функция является линейной по y . Поэтому минимум по y будет достигаться на одной из горизонтальных границ:

$$y = -50$$

или

$$y = 50.$$

Дополнительно проверим вертикальные границы:

$$x = -20$$

и

$$x = 20.$$

1. Нижняя граница $y = -50$

Подставим $y = -50$:

$$\begin{aligned} f(x, -50) &= 10^{-2} (8x^2 + 2x(-50) + 35x + 8(-50) + 12) \\ &= 10^{-2} (8x^2 - 100x + 35x - 400 + 12) \\ &= 10^{-2} (8x^2 - 65x - 388). \end{aligned}$$

Получили квадратичную функцию:

$$f(x, -50) = 10^{-2} (8x^2 - 65x - 388).$$

Так как коэффициент при x^2 положительный, парабола направлена вверх. Минимум достигается в вершине:

$$x = \frac{-b}{2a} = \frac{65}{2 \cdot 8} = \frac{65}{16} = 4.0625.$$

Точка принадлежит области:

$$4.0625 \in [-20, 20].$$

Вычислим значение функции:

$$\begin{aligned} f(4.0625, -50) &= 10^{-2} \left(8 \cdot 4.0625^2 + 2 \cdot 4.0625 \cdot (-50) \right. \\ &\quad \left. + 35 \cdot 4.0625 + 8 \cdot (-50) + 12 \right) \\ &= -5.2003125. \end{aligned}$$

На нижней границе кандидат на минимум:

$$(4.0625, -50), \quad f = -5.2003125.$$

2. Верхняя граница $y = 50$

Подставим $y = 50$:

$$\begin{aligned} f(x, 50) &= 10^{-2} (8x^2 + 2x \cdot 50 + 35x + 8 \cdot 50 + 12) \\ &= 10^{-2} (8x^2 + 100x + 35x + 400 + 12) \\ &= 10^{-2} (8x^2 + 135x + 412). \end{aligned}$$

Получили квадратичную функцию:

$$f(x, 50) = 10^{-2} (8x^2 + 135x + 412).$$

Так как коэффициент при x^2 положительный, парабола направлена вверх. Минимум достигается в вершине:

$$x = \frac{-135}{2 \cdot 8} = -\frac{135}{16} = -8.4375.$$

Точка принадлежит области:

$$-8.4375 \in [-20, 20].$$

Вычислим значение функции:

$$\begin{aligned} f(-8.4375, 50) &= 10^{-2} \left(8 \cdot (-8.4375)^2 + 2 \cdot (-8.4375) \cdot 50 \right. \\ &\quad \left. + 35 \cdot (-8.4375) + 8 \cdot 50 + 12 \right) \\ &= -1.5753125. \end{aligned}$$

На верхней границе кандидат на минимум:

$$(-8.4375, 50), \quad f = -1.5753125.$$

3. Левая граница $x = -20$

Подставим $x = -20$:

$$\begin{aligned} f(-20, y) &= 10^{-2} (8 \cdot (-20)^2 + 2 \cdot (-20)y + 35 \cdot (-20) + 8y + 12) \\ &= 10^{-2} (3200 - 40y - 700 + 8y + 12) \\ &= 10^{-2} (2512 - 32y). \end{aligned}$$

Получили линейную функцию по y :

$$f(-20, y) = 10^{-2} (2512 - 32y).$$

Коэффициент при y отрицательный, поэтому функция уменьшается при увеличении y . Значит, минимум на этой границе достигается при:

$$y = 50.$$

Вычислим значение:

$$f(-20, 50) = 10^{-2} (2512 - 32 \cdot 50) = 9.12.$$

На левой границе кандидат на минимум:

$$(-20, 50), \quad f = 9.12.$$

4. Правая граница $x = 20$

Подставим $x = 20$:

$$\begin{aligned} f(20, y) &= 10^{-2} (8 \cdot 20^2 + 2 \cdot 20y + 35 \cdot 20 + 8y + 12) \\ &= 10^{-2} (3200 + 40y + 700 + 8y + 12) \\ &= 10^{-2} (3912 + 48y). \end{aligned}$$

Получили линейную функцию по y :

$$f(20, y) = 10^{-2} (3912 + 48y).$$

Коэффициент при y положительный, поэтому функция увеличивается при увеличении y . Значит, минимум на этой границе достигается при:

$$y = -50.$$

Вычислим значение:

$$f(20, -50) = 10^{-2}(3912 + 48 \cdot (-50)) = 15.12.$$

На правой границе кандидат на минимум:

$$(20, -50), \quad f = 15.12.$$

Сравнение кандидатов на минимум

Сравним все найденные значения в таблице.

Таблица 1: Сравнение кандидатов на глобальный минимум

Граница	Кандидатная точка	Значение функции
Нижняя граница $y = -50$	$(4.0625, -50)$	-5.2003125
Верхняя граница $y = 50$	$(-8.4375, 50)$	-1.5753125
Левая граница $x = -20$	$(-20, 50)$	9.12
Правая граница $x = 20$	$(20, -50)$	15.12

Из таблицы видно, что наименьшее значение функции:

$$-5.2003125.$$

Оно достигается на нижней границе области в точке:

$$(4.0625, -50).$$

Следовательно, глобальный минимум функции на области:

$$[-20, 20] \times [-50, 50]$$

равен:

$$f_{\min} = -5.2003125$$

и достигается в точке:

$$(x_{\min}, y_{\min}) = (4.0625, -50).$$

Программная проверка минимума и максимума

Кроме аналитического решения, минимум и максимум можно проверить программно. Для этого строится сетка точек на заданной области, затем вычисляются значения функции на всей сетке. После этого выбираются точки с наименьшим и наибольшим значением функции.

```
import numpy as np

def f(x, y):
    return 10*(-2) * (8*x**2 + 2*x*y + 35*x + 8*y + 12)

x = np.linspace(-20, 20, 1000)
y = np.linspace(-50, 50, 1000)

X, Y = np.meshgrid(x, y)
Z = f(X, Y)

min_index = np.unravel_index(np.argmin(Z), Z.shape)
max_index = np.unravel_index(np.argmax(Z), Z.shape)

x_min = X[min_index]
y_min = Y[min_index]
f_min = Z[min_index]

x_max = X[max_index]
y_max = Y[max_index]
f_max = Z[max_index]

print("Minimum:", x_min, y_min, f_min)
print("Maximum:", x_max, y_max, f_max)
```

Такой код дает приближенную численную проверку аналитического результата: минимум находится около точки $(4.0625, -50)$, а максимум — в точке $(20, 50)$.

Линии уровня

Для визуальной проверки результата были построены линии уровня функции. На графике отмечены седловая точка и точка глобального минимума.

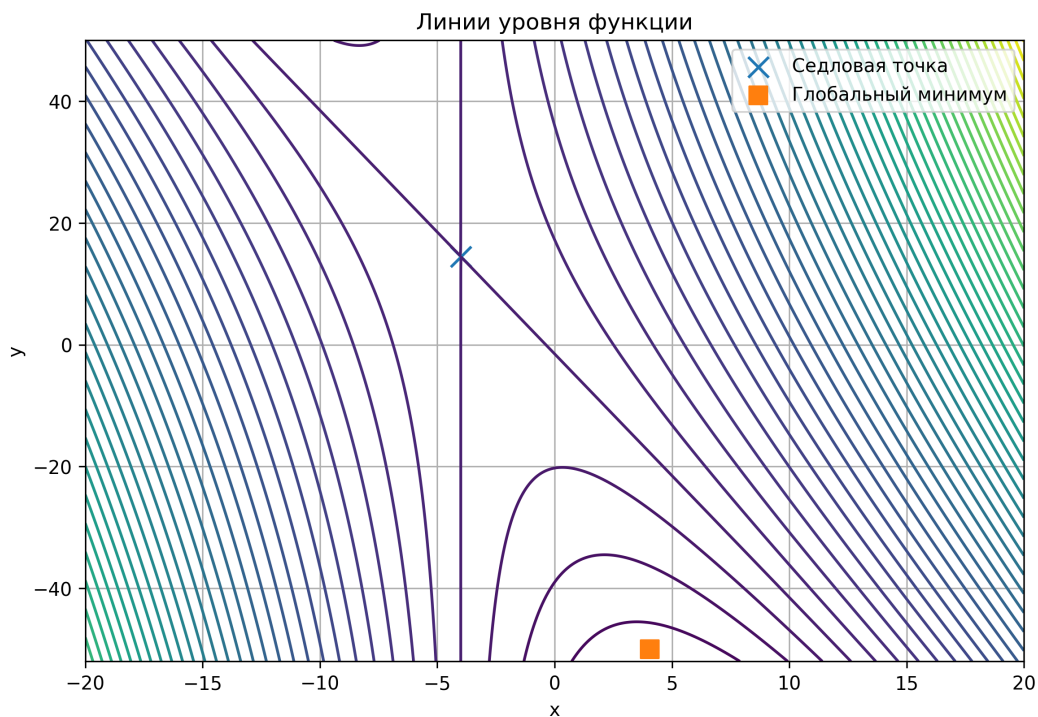


Рис. 1: Линии уровня функции варианта 7

Вывод по заданию 1

Функция имеет одну стационарную точку $(-4, 14.5)$. Так как определитель гессиана отрицателен, эта точка является седловой. Локальных экстремумов внутри области нет. Глобальный минимум достигается на границе области в точке:

$$(4.0625, -50).$$

Сложность для оптимизационного алгоритма состоит в том, что около седловой точки градиент может быть малым. Из-за этого обычный градиентный спуск может замедлиться около седловой точки, хотя она не является минимумом.

2 Задание 2. Преодоление седловой точки

Теоретическое описание проблемы

Обычный градиентный спуск использует формулу:

$$x_{k+1} = x_k - \alpha \nabla f(x_k),$$

где x_k — текущая точка, α — скорость обучения, $\nabla f(x_k)$ — градиент функции.

Если точка находится около седловой точки, то градиент может быть очень малым:

$$\nabla f(x_k) \approx 0.$$

Тогда шаг:

$$-\alpha \nabla f(x_k)$$

становится маленьким, и алгоритм может замедлиться. Проблема в том, что седловая точка не является минимумом, но обычный градиентный спуск может вести себя так, будто он почти пришел к решению.

В коде функция варианта 7 и ее градиент задаются так:

```
def f(x, y):  
    return 10**(-2) * (8*x**2 + 2*x*y + 35*x + 8*y + 12)  
  
def grad(x, y):  
    fx = 10**(-2) * (16*x + 2*y + 35)  
    fy = 10**(-2) * (2*x + 8)  
    return np.array([fx, fy])
```

Седловая точка была найдена аналитически в задании 1:

$$(-4, 14.5).$$

В коде она задается явно:

```
saddle = np.array([-4.0, 14.5])
```

Выбор начального приближения через собственные векторы гессиана

В задании 1 была найдена седловая точка:

$$x^* = (-4, 14.5).$$

Седловую точку мы не меняем: она является свойством самой функции. В задании 2 нужно выбрать такое начальное приближение, чтобы обычный градиентный спуск за фиксированное число итераций пришел в окрестность этой седловой точки.

Для выбора начального приближения рассмотрим гессиан функции:

$$H = \begin{pmatrix} 0.16 & 0.02 \\ 0.02 & 0 \end{pmatrix}.$$

Найдем собственные значения гессиана из уравнения:

$$\det(H - \lambda I) = 0.$$

Имеем:

$$H - \lambda I = \begin{pmatrix} 0.16 - \lambda & 0.02 \\ 0.02 & -\lambda \end{pmatrix}.$$

Тогда:

$$\det(H - \lambda I) = (0.16 - \lambda)(-\lambda) - 0.02^2.$$

Раскроем скобки:

$$(0.16 - \lambda)(-\lambda) - 0.0004 = -0.16\lambda + \lambda^2 - 0.0004.$$

Получаем характеристическое уравнение:

$$\lambda^2 - 0.16\lambda - 0.0004 = 0.$$

Решим квадратное уравнение:

$$D = (-0.16)^2 - 4 \cdot 1 \cdot (-0.0004).$$

$$D = 0.0256 + 0.0016 = 0.0272.$$

$$\sqrt{D} \approx 0.164924.$$

Тогда:

$$\lambda_{1,2} = \frac{0.16 \pm 0.164924}{2}.$$

Получаем:

$$\lambda_1 \approx 0.162462,$$

$$\lambda_2 \approx -0.002462.$$

Так как одно собственное значение положительное, а другое отрицательное, это еще раз подтверждает, что стационарная точка является седловой.

Теперь найдем собственные векторы.

Для положительного собственного значения:

$$\lambda_1 \approx 0.162462$$

решим систему:

$$(H - \lambda_1 I)v = 0.$$

Пусть:

$$v = (v_1, v_2).$$

Тогда из первой строки системы:

$$(0.16 - \lambda_1)v_1 + 0.02v_2 = 0.$$

Подставим $\lambda_1 \approx 0.162462$:

$$(0.16 - 0.162462)v_1 + 0.02v_2 = 0.$$

$$-0.002462v_1 + 0.02v_2 = 0.$$

Отсюда:

$$0.02v_2 = 0.002462v_1.$$

$$v_2 = \frac{0.002462}{0.02}v_1.$$

$$v_2 \approx 0.1231v_1.$$

Возьмем:

$$v_1 = 1.$$

Тогда:

$$v_2 \approx 0.1231.$$

Значит, собственный вектор для положительного собственного значения можно взять в виде:

$$v^{(1)} = (1, 0.1231).$$

Это направление является устойчивым для обычного градиентного спуска в окрестности седловой точки. Если начальная точка расположена вдоль этого направления, то обычный градиентный спуск движется к седловой точке.

Теперь найдем собственный вектор для отрицательного собственного значения:

$$\lambda_2 \approx -0.002462.$$

Снова решаем систему:

$$(H - \lambda_2 I)v = 0.$$

Из первой строки:

$$(0.16 - \lambda_2)v_1 + 0.02v_2 = 0.$$

Подставим $\lambda_2 \approx -0.002462$:

$$(0.16 + 0.002462)v_1 + 0.02v_2 = 0.$$

$$0.162462v_1 + 0.02v_2 = 0.$$

Отсюда:

$$0.02v_2 = -0.162462v_1.$$

$$v_2 = \frac{-0.162462}{0.02}v_1.$$

$$v_2 \approx -8.1231v_1.$$

Возьмем:

$$v_1 = 1.$$

Тогда:

$$v_2 \approx -8.1231.$$

Значит, собственный вектор для отрицательного собственного значения можно взять в виде:

$$v^{(2)} = (1, -8.1231).$$

Это направление является неустойчивым. Малое смещение в этом направлении нужно для того, чтобы метод Поляка мог не просто прийти к седловой точке, а пройти через ее окрестность и уйти в сторону меньших значений функции.

Поэтому начальное приближение выберем как сумму двух слагаемых:

$$x_0 = x^* + t_1v^{(1)} + t_2v^{(2)}.$$

Здесь $v^{(1)}$ задает основное движение к седловой точке, а $v^{(2)}$ задает небольшое смещение от точного устойчивого направления.

Возьмем:

$$t_1 = -9.9242, \quad t_2 = 0.0122.$$

Тогда:

$$(x_0, y_0) = (-4, 14.5) - 9.9242(1, 0.1231) + 0.0122(1, -8.1231).$$

Вычислим координату x_0 :

$$x_0 = -4 - 9.9242 + 0.0122.$$

$$x_0 = -13.912.$$

Вычислим координату y_0 :

$$y_0 = 14.5 - 9.9242 \cdot 0.1231 + 0.0122 \cdot (-8.1231).$$

$$y_0 \approx 13.179.$$

Получаем начальное приближение:

$$(x_0, y_0) = (-13.912, 13.179).$$

Таким образом, начальная точка выбрана не произвольно. Она находится почти вдоль устойчивого направления седловой точки, но имеет небольшое смещение в неустойчивом направлении. Благодаря этому обычный градиентный спуск за 100 итераций приходит близко к седловой точке, а метод Поляка за счет момента может преодолеть ее окрестность и продолжить движение дальше.

Метод Поляка и его смысл

По варианту 7 используется SGD с моментом, то есть модификация Поляка.

Формула метода Поляка:

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1}).$$

Здесь:

- α — скорость обучения;
- β — коэффициент момента;
- $-\alpha \nabla f(x_k)$ — обычный шаг градиентного спуска;
- $\beta(x_k - x_{k-1})$ — момент, то есть учет предыдущего направления движения.

Смысл метода Поляка состоит в том, что алгоритм не только смотрит на текущий градиент, но и запоминает, куда он двигался раньше. Если около седловой точки градиент становится маленьким, обычный градиентный спуск почти останавливается. Метод Поляка за счет момента продолжает движение в прежнем направлении и может пройти через окрестность седловой точки.

Сравнение шага обычного градиентного спуска и метода Поляка

Таблица 2: Сравнение шага обычного градиентного спуска и метода Поляка

Метод	Формула шага	Особенность
Обычный градиентный спуск	$x_{k+1} = x_k - \alpha \nabla f(x_k)$	Использует только текущий градиент. В окрестности седловой точки может замедлиться, так как градиент мал.
Метод Поляка	$x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1})$	Использует текущий градиент и момент. За счет момента может пройти через седловую точку.

2.1 Обычный градиентный спуск

В пункте 2.1 был реализован обычный градиентный спуск. Его задача — показать, что при выбранном начальном приближении алгоритм может приблизиться к седловой точке.

Формула:

$$x_{k+1} = x_k - \alpha \nabla f(x_k).$$

В коде это реализовано следующим образом:

```
def gradient_descent(x0, alpha, n_iter):
    x = x0.copy()
    history = [x.copy()]

    for i in range(n_iter):
        g = grad(x[0], x[1])
        x = x - alpha * g
        history.append(x.copy())

    return np.array(history)
```

Начальное приближение:

$$(x_0, y_0) = (-13.912, 13.179).$$

Оно было подобрано экспериментально. Цель подбора состояла в том, чтобы обычный градиентный спуск примерно за 100 итераций пришел в окрестность седловой точки:

$$(-4, 14.5).$$

Параметры:

$$\alpha = 0.5, \quad n = 100.$$

В коде запуск выглядит так:

```
x0 = np.array([-13.912, 13.179])
alpha_gd = 0.5
n_iter = 100

gd_history = gradient_descent(x0, alpha_gd, n_iter)
```

В этой лабораторной работе в качестве критерия останова было зафиксировано число итераций:

$$n = 100.$$

Это нужно для того, чтобы все последующие методы сравнивались на одинаковом числе шагов.

После 100 итераций получена точка:

$$(-3.98825551, 14.38747522).$$

Седловая точка:

$$(-4, 14.5).$$

Расстояние до седловой точки:

$$0.11313602025031912.$$

Норма градиента:

$$0.00043942520408789954.$$

Значение функции:

$$f = -1.5396278504056225 \cdot 10^{-5}.$$

Так как итоговая точка близка к седловой точке, а норма градиента мала, можно сделать вывод, что обычный градиентный спуск приблизился к седловой точке и замедлился около нее.

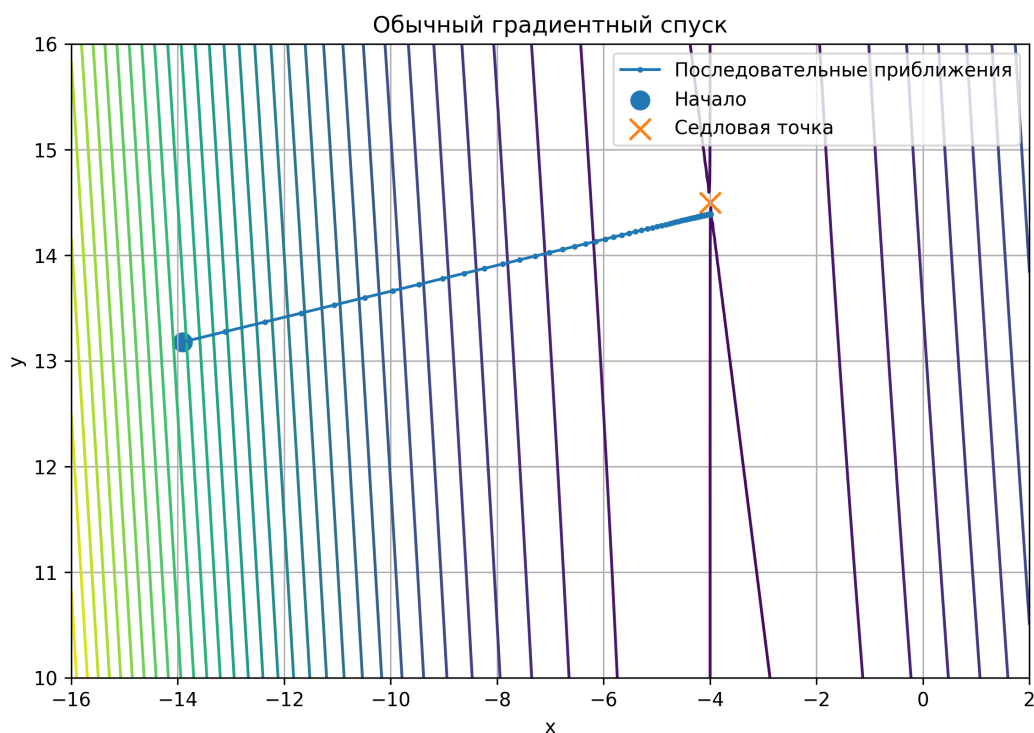


Рис. 2: Обычный градиентный спуск: движение к седловой точке

2.2 Ручная реализация метода Поляка

В пункте 2.2 был реализован метод Поляка. Его задача — показать, что метод с моментом может не остановиться около седловой точки, а продолжить движение в сторону меньших значений функции.

Формула метода:

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1}).$$

В коде метод Поляка реализован так:

```
def polyak_method(x0, alpha, beta, n_iter):
    x_prev = x0.copy()
    x = x0.copy()

    history = [x.copy()]

    for i in range(n_iter):
        g = grad(x[0], x[1])
        x_next = x - alpha * g + beta * (x - x_prev)

        x_prev = x.copy()
        x = x_next.copy()

        history.append(x.copy())

    return np.array(history)
```


В строке:

```
x_next = x - alpha * g + beta * (x - x_prev)
```

первая часть

$$x - \alpha g$$

соответствует обычному градиентному шагу, а часть

$$\beta(x - x_{\text{prev}})$$

добавляет момент, то есть предыдущее направление движения.

Параметры:

$$\alpha = 2.5, \quad \beta = 0.95, \quad n = 100.$$

Запуск в коде:

```
alpha_polyak = 2.5
beta_polyak = 0.95

polyak_history = polyak_method(
    x0,
    alpha_polyak,
    beta_polyak,
    n_iter
)
```

После 100 итераций получена точка:

$$(-0.53381717, -7.08346883).$$

Расстояние до седловой точки:

$$21.860021729389857.$$

Значение функции:

$$f = -0.5350911081150261.$$

По сравнению с обычным градиентным спуском метод Поляка ушел значительно дальше от седловой точки. Это означает, что алгоритм не остановился около седловой точки, а продолжил движение в область меньших значений функции.

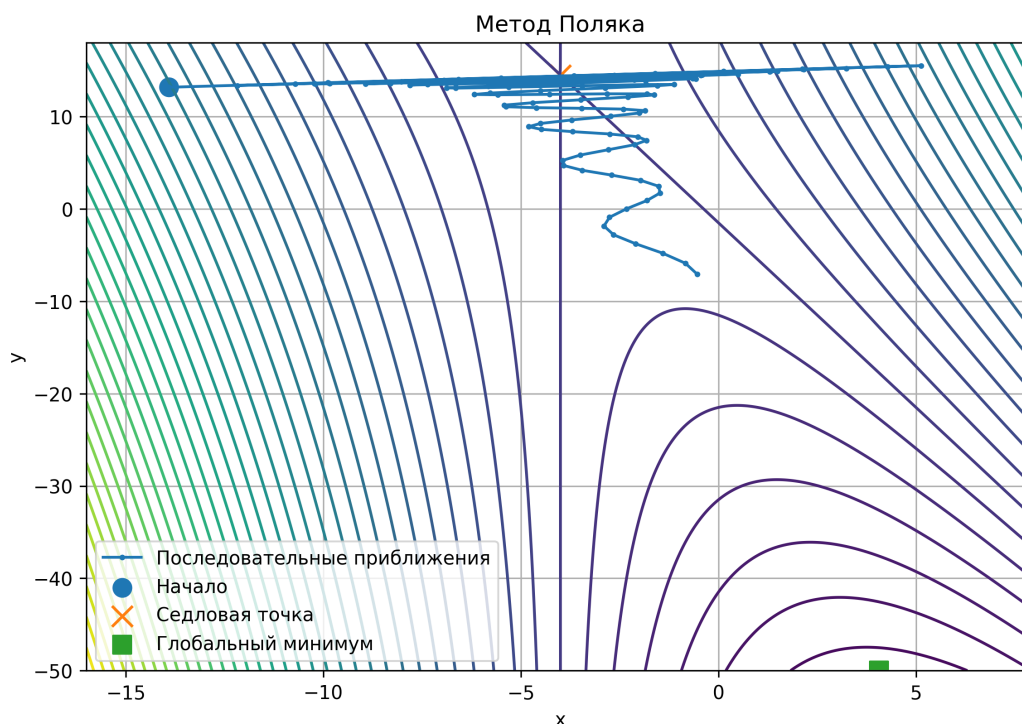


Рис. 3: Метод Поляка: преодоление седловой точки

2.3 Подбор гиперпараметров

В этом пункте сравнивались два набора гиперпараметров метода Поляка. Цель состояла в том, чтобы показать, что характер движения зависит от параметров α и β .

Пилообразная траектория

Первый набор:

$$\alpha = 3.0, \quad \beta = 0.95.$$

При таком выборе скорость обучения достаточно большая, поэтому траектория имеет более заметные боковые отклонения. Такое движение называется пилообразным, потому что последовательные приближения образуют более резкую ломаную.

Запуск в коде:

```
alpha_zigzag = 3.0
beta_zigzag = 0.95

zigzag_history = polyak_method(
    x0,
    alpha_zigzag,
    beta_zigzag,
    n_iter
)
```

Результат после 100 итераций:

$$(2.1926425, -29.44656622), \quad f = -2.375001783288437.$$

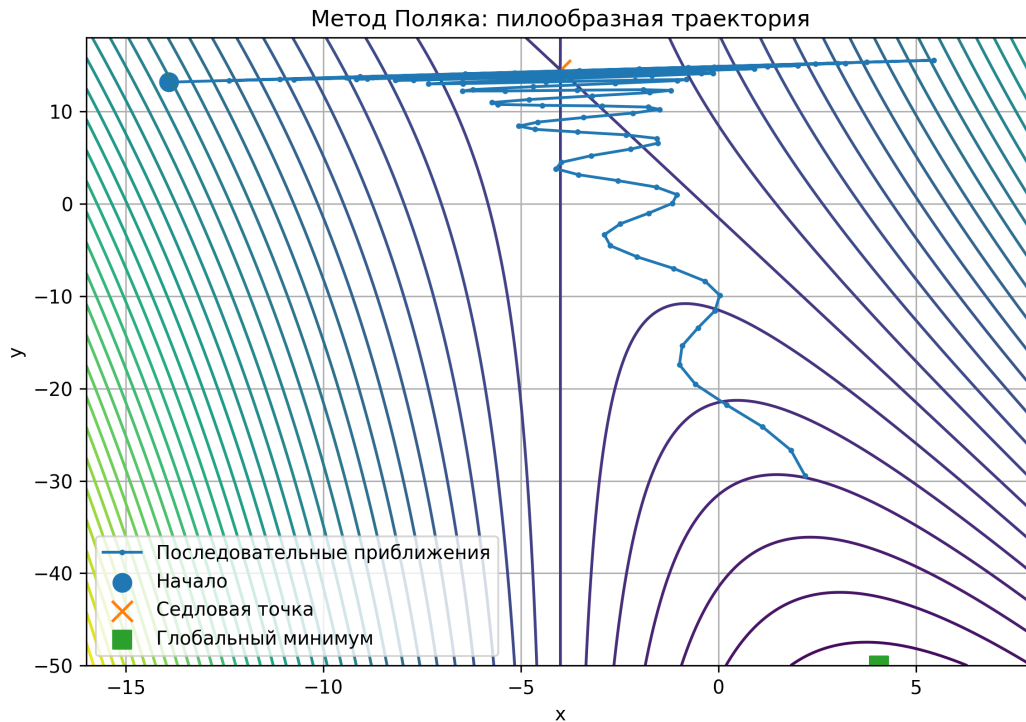


Рис. 4: Метод Поляка: пилообразная траектория

Более ровная траектория

Второй набор:

$$\alpha = 2.0, \quad \beta = 0.98.$$

При этих параметрах движение получается более направленным и визуально более ровным. Отклонения в стороны выражены слабее.

Запуск в коде:

```
alpha_smooth = 2.0
beta_smooth = 0.98

smooth_history = polyak_method(
    x0,
    alpha_smooth,
    beta_smooth,
    n_iter
)
```

Результат после 100 итераций:

$$(0.17479421, -11.76888571), \quad f = -0.7990313026772345.$$

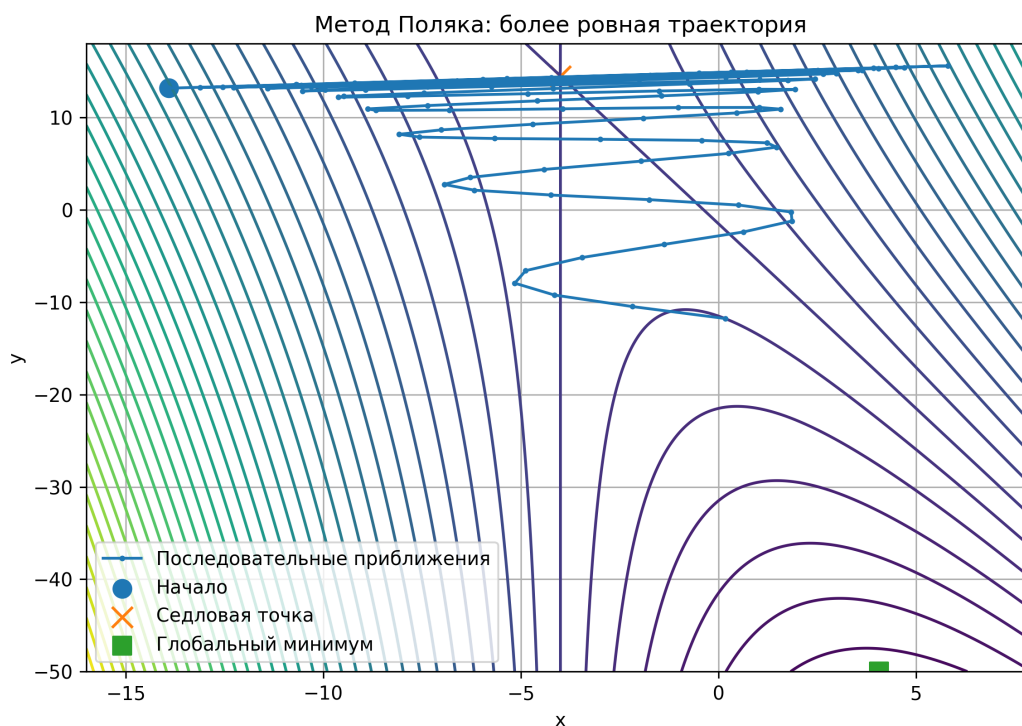


Рис. 5: Метод Поляка: более ровная траектория

Таблица 3: Сравнение пилообразной и более ровной траектории

Вариант	α	β	Описание
Пилообразная траектория	3.0	0.95	Более резкое движение, заметные боковые отклонения, быстрый уход в область меньших значений функции.
Более ровная траектория	2.0	0.98	Более спокойное движение, меньше визуальных отклонений в стороны.

Таблица 4: Итоговые результаты для двух траекторий

Вариант	Итоговая точка	Значение функции
Пилообразная траектория	(2.1926, -29.4466)	-2.375002
Более ровная траектория	(0.1748, -11.7689)	-0.799031

Пилообразная траектория за 100 итераций дала меньшее значение функции, то есть быстрее ушла в сторону глобального минимума. Однако такое

движение менее устойчиво визуально, так как сопровождается более резкими отклонениями. Более ровная траектория движется спокойнее, но за то же число итераций уменьшает значение функции слабее.

2.4 Готовая реализация PyTorch

В пункте 2.4 использована готовая реализация оптимизатора из библиотеки PyTorch:

`torch.optim.SGD`

с параметром:

`momentum.`

По смыслу это соответствует методу Поляка, так как при обновлении учитывается не только текущий градиент, но и накопленное направление движения.

В коде PyTorch-оптимизатор задается так:

```
optimizer = torch.optim.SGD(
    [z],
    lr=alpha,
    momentum=beta
)
```

Сам процесс оптимизации:

```
for i in range(n_iter + 1):
    history.append(z.detach().numpy().copy())

    optimizer.zero_grad()
    loss = torch_f(z)
    loss.backward()
    optimizer.step()
```

Здесь:

- `optimizer.zero_grad()` очищает старые градиенты;
- `loss = torch_f(z)` вычисляет значение функции;
- `loss.backward()` вычисляет градиент;
- `optimizer.step()` делает шаг оптимизатора.

Параметры:

$$\alpha = 2.0, \quad \beta = 0.98, \quad n = 100.$$

После 100 итераций PyTorch дал точку:

$$(0.17485285, -11.769379),$$

$$f = -0.7990641.$$

Для сравнения ручная реализация метода Поляка при тех же параметрах дала:

$$(0.17479421, -11.76888571),$$

$$f = -0.7990313026772345.$$

Таблица 5: Сравнение ручной реализации метода Поляка и PyTorch

Метод	Итоговая точка	Значение функции
Ручная реализация метода Поляка	(0.1748, -11.7689)	-0.799031
PyTorch SGD с momentum	(0.1749, -11.7694)	-0.799064

Результаты практически совпадают. Это подтверждает корректность ручной реализации метода Поляка.

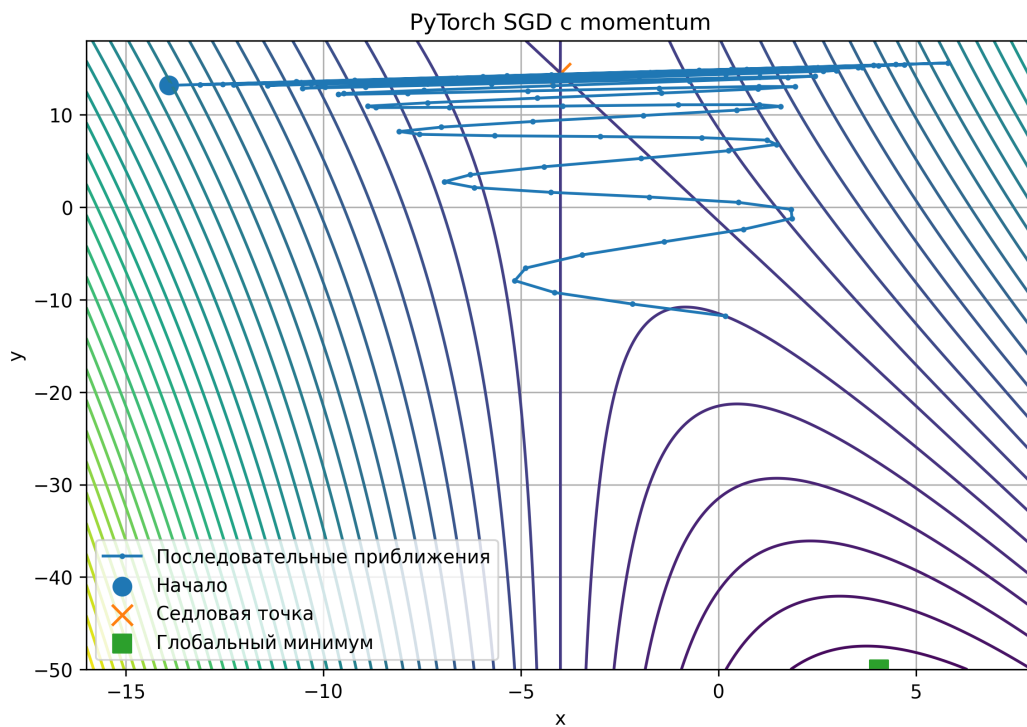


Рис. 6: PyTorch SGD с momentum: преодоление седловой точки

Результаты запуска программы

При запуске программы были получены следующие ключевые значения.

Таблица 6: Результаты методов после 100 итераций

Метод	Последняя точка	Значение функции
Обычный градиентный спуск	$(-3.9883, 14.3875)$	$-1.54 \cdot 10^{-5}$
Метод Поляка	$(-0.5338, -7.0835)$	-0.535091
Пилообразная траектория	$(2.1926, -29.4466)$	-2.375002
Более ровная траектория	$(0.1748, -11.7689)$	-0.799031
PyTorch SGD с momentum	$(0.1749, -11.7694)$	-0.799064

Седловая точка:

$$(-4, 14.5).$$

Обычный градиентный спуск приходит близко к седловой точке. Метод Поляка и PyTorch SGD с momentum уходят от нее вниз, то есть преодолевают окрестность седловой точки.

Вывод по заданию 2

Обычный градиентный спуск приблизился к седловой точке и замедлился около нее. Метод Поляка, благодаря слагаемому момента, смог пройти через окрестность седловой точки. Подбор гиперпараметров показал, что при разных значениях α и β меняется характер движения: траектория может быть более пилообразной или более ровной. Готовая реализация PyTorch дала почти тот же результат, что и ручная реализация.

3 Задание 3. Обучение нейронной сети

3.1 Описание датасета и решаемой задачи

Для задания 3 был выбран датасет **PhiUSIIL Phishing URL Dataset** из UCI Machine Learning Repository.

Это табличный датасет для задачи бинарной классификации URL-адресов. Каждый объект в датасете — это один URL-адрес. Признаки описывают характеристики URL, а целевая переменная показывает, является ли URL легитимным или фишинговым.

Решается задача бинарной классификации:

$$y \in \{0, 1\},$$

где

$$y = 1$$

— легитимный URL, а

$$y = 0$$

— фишинговый URL.

Основная информация о датасете приведена в таблице.

Таблица 7: Описание датасета

Характеристика	Значение
Название датасета	PhiUSIIL Phishing URL Dataset
Тип задачи	Бинарная классификация
Количество объектов	235795
Исходное количество признаков	54
Количество числовых признаков после отбора	50
Класс 1	134850 объектов
Класс 0	100945 объектов

Таким образом, на вход модели подается вектор из 50 числовых признаков URL-адреса, а на выходе модель должна определить класс объекта: легитимный URL или фишинговый URL.

3.2 Этапы предобработки данных

Перед обучением данные были подготовлены следующим образом:

1. датасет был загружен из UCI Machine Learning Repository;
2. целевая переменная была приведена к целочисленному типу;
3. из признаков были оставлены только числовые признаки;
4. данные были проверены на пропуски;
5. выборка была разделена на обучающую и тестовую в пропорции 80/20;
6. числовые признаки были стандартизированы.

Стандартизация выполнялась по формуле:

$$x' = \frac{x - \mu}{\sigma},$$

где μ — среднее значение признака, σ — стандартное отклонение.

Стандартизация нужна для того, чтобы все признаки имели сопоставимый масштаб. Это важно для градиентных методов, потому что при сильно разных масштабах признаков обучение может идти медленнее или нестабильнее.

3.3 Архитектура нейронной сети

Для решения задачи была построена полносвязная нейронная сеть. После предобработки осталось 50 числовых признаков, поэтому входной слой имеет размерность 50.

Архитектура сети:

$$50 \rightarrow 64 \rightarrow 32 \rightarrow 1.$$

В коде сеть задана следующим образом:

```
self.net = nn.Sequential(  
    nn.Linear(input_size, 64),  
    nn.ReLU(),  
  
    nn.Linear(64, 32),  
    nn.ReLU(),  
  
    nn.Linear(32, 1)  
)
```

Используемые слои:

- **Linear** — полносвязный слой. В нем есть обучаемые веса и смещения.
- **ReLU** — функция активации. Она не имеет обучаемых весов, но добавляет нелинейность.

Первый слой:

Linear(50, 64)

получает 50 входных признаков и преобразует их в 64 значения.

Второй слой:

Linear(64, 32)

преобразует 64 значения в 32.

Выходной слой:

Linear(32, 1)

выдает одно число, потому что решается задача бинарной классификации.

Это число является логитом. Чтобы получить вероятность класса 1, используется сигмоида:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Если:

$$\sigma(z) \geq 0.5,$$

то объект относится к классу 1. Если:

$$\sigma(z) < 0.5,$$

то объект относится к классу 0.

3.4 Целевая функция

У нас задача классификации, причем бинарной, поэтому используется бинарная кросс-энтропия.

Для одного объекта бинарная кросс-энтропия имеет вид:

$$L(y, \hat{y}) = -(y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})),$$

где y — истинный класс объекта, а $\hat{y}$ — предсказанная вероятность класса 1.

Для всей выборки функция потерь записывается так:

$$L = -\frac{1}{N} \sum_{i=1}^N (y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)).$$

В коде эта функция потерь реализована строкой:

```
criterion = nn.BCEWithLogitsLoss()
```

Далее она используется при обучении:

```
logits = model(X_batch)
loss = criterion(logits, y_batch)
```

Функция `BCEWithLogitsLoss` удобна тем, что она внутри объединяет сигмоиду и бинарную кросс-энтропию. Поэтому в последнем слое сети не нужно явно добавлять `Sigmoid`. Модель выдает логит, а функция потерь сама корректно преобразует его при вычислении ошибки.

3.5 Обучение модели

Обучение нейронной сети происходит следующим образом. Сначала модель получает батч объектов, затем делает предсказание, после этого считается значение функции потерь. Далее PyTorch вычисляет градиенты ошибки по весам, и оптимизатор обновляет веса модели.

Основной фрагмент обучения:

```
optimizer.zero_grad()

logits = model(X_batch)
```

```
loss = criterion(logits, y_batch)

loss.backward()
optimizer.step()
```

Здесь:

- `optimizer.zero_grad()` обнуляет старые градиенты;
- `model(X_batch)` получает предсказания модели;
- `criterion(logits, y_batch)` считает ошибку;
- `loss.backward()` вычисляет градиенты;
- `optimizer.step()` обновляет веса модели.

Параметры обучения:

Таблица 8: Параметры обучения нейронной сети

Параметр	Значение
Скорость обучения α	0.01
Коэффициент момента β	0.9
Размер батча	512
Количество эпох	10
Разбиение train/test	80/20

3.6 Оптимизаторы

В работе сравнивались два оптимизатора.

Первый оптимизатор — готовая реализация PyTorch:

`torch.optim.SGD`

с параметром:

`momentum.`

Код:

```
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.01,
    momentum=0.9
)
```

Второй оптимизатор — собственная реализация метода Поляка. Она была реализована в классе:

PolyakSGD.

Формула метода Поляка для весов нейронной сети:

$$w_{k+1} = w_k - \alpha \nabla E(w_k) + \beta(w_k - w_{k-1}),$$

где w_k — веса нейронной сети на текущем шаге, $E(w_k)$ — функция ошибки, α — скорость обучения, β — коэффициент момента.

В коде обновление весов реализовано так:

```
p.add_(p.grad, alpha=-lr)
p.add_(current_param - prev_param, alpha=beta)
```

Первая строка выполняет обычный шаг градиентного спуска:

$$w_k - \alpha \nabla E(w_k).$$

Вторая строка добавляет момент:

$$\beta(w_k - w_{k-1}).$$

То есть собственный оптимизатор использует ту же идею, что и метод Поляка из задания 2: учитывается не только текущий градиент, но и предыдущее направление движения.

3.7 Результаты обучения

Для оценки качества использовались следующие метрики:

- **Accuracy** — доля правильных ответов;
- **Precision** — точность положительных предсказаний;
- **Recall** — полнота;
- **F1** — среднее гармоническое между Precision и Recall.

Итоговые результаты на тестовой выборке представлены в таблице.

Таблица 9: Сравнение библиотечного и собственного оптимизатора

Оптимизатор	Loss	Accuracy	Precision	Recall	F1
PyTorch SGD momentum	0.000748	0.999767	0.999666	0.999926	0.999796
Custom PolyakSGD	0.000749	0.999767	0.999666	0.999926	0.999796

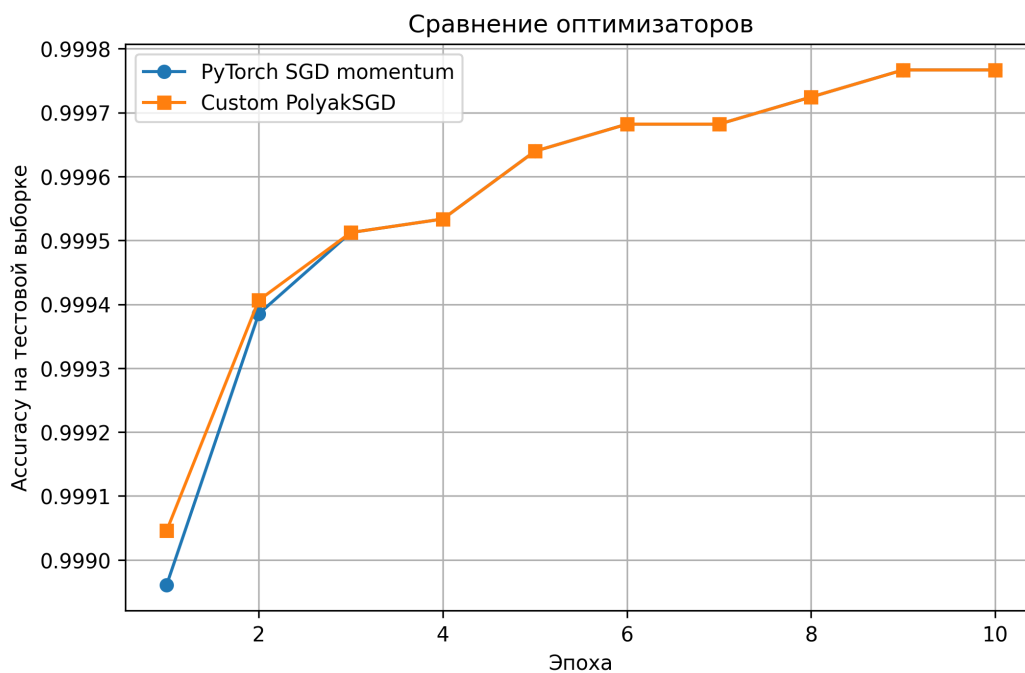


Рис. 7: Сравнение Ассурасу для двух оптимизаторов

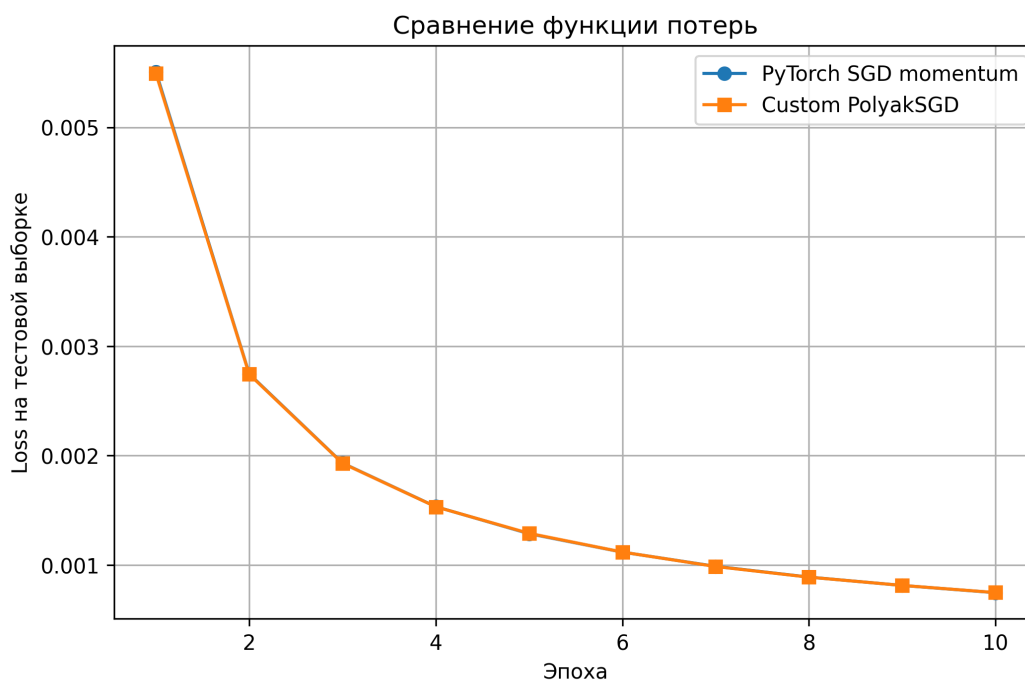


Рис. 8: Сравнение функции потерь для двух оптимизаторов

3.8 Анализ графиков и результатов

На графике Ассурасу видно, что оба оптимизатора быстро достигают высокого качества на тестовой выборке. Кривые для PyTorch SGD с momentum и Custom PolyakSGD практически совпадают. Это означает, что собственная реализация обучает модель почти так же, как библиотечная.

На графике Loss видно, что значение функции потерь уменьшается от эпохи к эпохе для обоих оптимизаторов. Это означает, что обучение проходит корректно: модель постепенно уменьшает ошибку.

Финальные значения функции потерь:

$$L_{\text{PyTorch}} = 0.000748,$$

$$L_{\text{Custom}} = 0.000749.$$

Разница:

$$0.000749 - 0.000748 = 0.000001.$$

Эта разница очень мала. Поэтому можно считать, что оба оптимизатора работают практически одинаково.

Ассурасу для обоих методов:

$$0.999767.$$

F1-мера для обоих методов:

$$0.999796.$$

Это означает, что модель правильно классифицирует примерно 99.98% объектов тестовой выборки. Высокое значение F1 также показывает, что модель хорошо работает с обоими классами, а не просто угадывает наиболее частый класс.

В данной задаче лучше считается тот метод, у которого меньше значение функции потерь и выше значения Ассурасу, Precision, Recall и F1. По этим критериям методы практически равны. Библиотечная реализация имеет немного меньший Loss, но различие настолько мало, что существенного преимущества нет.

Вывод по заданию 3

В задании 3 была решена задача бинарной классификации URL-адресов. Нейронная сеть получала на вход 50 числовых признаков URL и должна была определить, является URL легитимным или фишинговым.

Для обучения использовались два оптимизатора: готовый PyTorch SGD с momentum и собственная реализация PolyakSGD. Собственная реализация была основана на формуле метода Поляка и была встроена в процесс обучения нейронной сети.

Результаты двух оптимизаторов практически совпали. Это подтверждает, что собственная реализация метода Поляка работает корректно и дает качество, сопоставимое с библиотечной реализацией PyTorch.

4 Общий вывод

В ходе лабораторной работы была исследована функция варианта 7:

$$f(x, y) = 10^{-2} (8x^2 + 2xy + 35x + 8y + 12) .$$

Была найдена седловая точка:

$$(-4, 14.5).$$

Глобальный минимум на заданной области достигается на границе:

$$(4.0625, -50), \quad f_{\min} = -5.2003125.$$

Обычный градиентный спуск был настроен так, чтобы приблизиться к седловой точке за 100 итераций. Это показало проблему седловых точек: алгоритм может замедлиться около точки, которая не является минимумом.

Метод Поляка использует момент:

$$\beta(x_k - x_{k-1}),$$

поэтому он учитывает предыдущее направление движения и может пройти через окрестность седловой точки.

В третьем задании метод Поляка был применен уже не к функции двух переменных, а к обучению нейронной сети. Для задачи классификации фишинговых URL собственная реализация PolyakSGD показала практически такое же качество, как библиотечный оптимизатор PyTorch SGD с momentum.

Таким образом, цель работы достигнута: была показана проблема седловых точек и продемонстрировано, что модификация градиентного спуска с моментом помогает преодолевать такие области как на искусственной функции, так и при обучении нейронной сети.